

# Module Réseaux et Systèmes Avancés (RSA) – Partie Réseau

Rémi Badonnel  
Slides d'Isabelle Chrisment



1

1

## Module RSA – Partie Réseau

- **Protocoles de Routage**
  - Routing Information Protocol (RIP)
  - Open Shortest Path First (OSPF)
- **Protocoles de Transport**
  - Contrôle de congestion et TCP
- **Programmation Réseaux**
  - Utilisation de sockets
- **Introduction à IPv6**



2

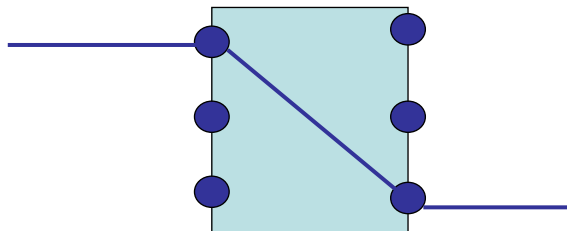
2

# Protocoles de Routage : *Routing Information Protocol (RIP)*

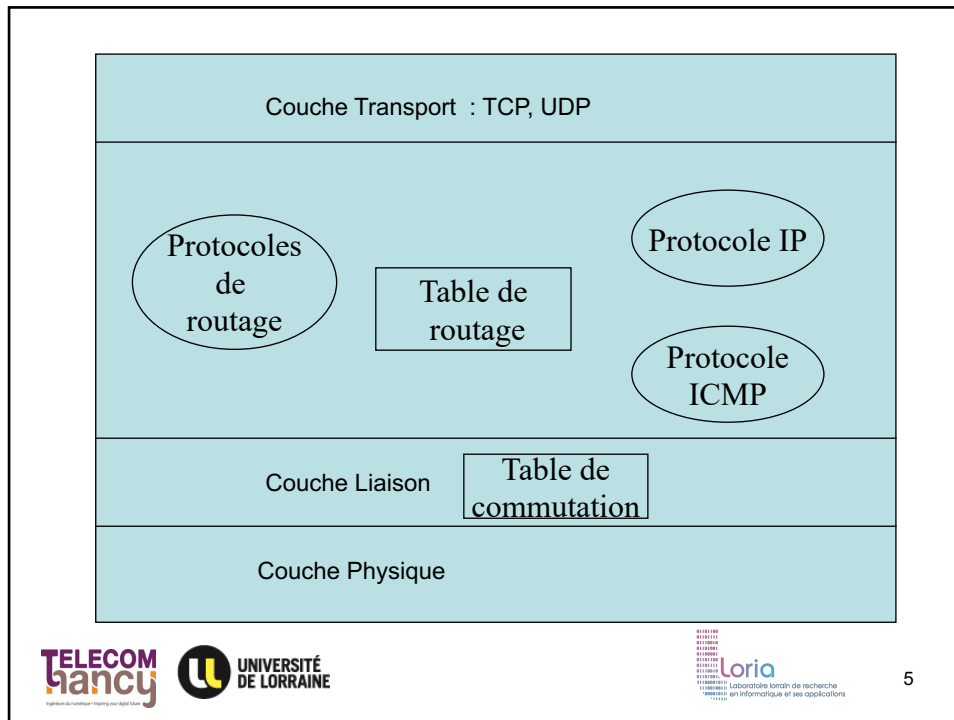
Rémi Badonnell  
Slides d'Isabelle Chrisment

## Le Routage

- commutation et routage
  - commutation : sélectionner un port de sortie en fonction de l'adresse destination et de la table de routage



- routage : process par lequel la table de routage est construite



5

## Le Routage

- Le réseau est vu comme un graphe :
  - Les nœuds → les routeurs
  - Les arêtes → les liaisons physiques
  - La valeur de l'arête → le coût pour envoyer un paquet sur la liaison
- Problème : trouver le chemin de coût minimal entre deux nœuds :
  - Le coût : le niveau de congestion, la distance physique

6

6

# Le Routage

- Routage statique
  - Routage manuel
  - Commande “route” sous unix
- Routage dynamique
  - Routage automatique et adaptatif
  - Echange d’informations de routage via des protocoles de routage

# Route statique

- Commande route
  - Permet d’indiquer une route
    - Vers une réseau (net) ou vers un équipement (host)
    - Ou une route par défaut (default)
  - Syntaxe
    - **route add|delete[net|host]destination|default gateway metric**
    - Sur les équipements non routeurs, une seule route par défaut est définie

## Route statique

- Commande `ifconfig/ip addr`
  - Permet de configurer une interface en lui attribuant une adresse IP
  - Syntaxe
    - `ifconfig eth0 @IP netmask @mask broadcast @cast`
    - `ip addr add @IP/@mask dev eth0`
- Consultation des routes :
  - Commande `route -n`

## Problèmes du routage statique

- Statique implique
  - Mise à jour manuelle de tous les équipements réseaux
  - Difficile à maintenir en cas d'évolution du réseau
- Recommandation
  - Stations ➔ routage statique
  - Routeurs ➔ routage dynamique

## Routage dynamique

- Est basé sur des protocoles de routage qui appliquent des algorithmes permettant la mise à jour automatique des tables de routage
- Principe :
  - Diffusion périodique de messages liés au routage sur le réseau

## Algorithmes de Routage

- Deux algorithmes :
  - Vecteur de Distance
    - Un nœud dit à ses voisins sa distance par rapport à tous les autres nœuds du réseau.
  - Etat de Liaison
    - Un nœud dit à tous les autres nœuds du réseau sa distance par rapport à ses voisins

## Algorithme de Routage Vecteur de Distance

- Algorithme :
  - Itératif
    - Continue tant qu'il y a des informations à échanger
  - Asynchrone
  - Distribué
    - Aucun nœud n'a la vision complète du réseau

## Vecteur de Distance

- Chaque nœud maintient par destination
  - <Destination, Coût, Prochain Nœud> → table de routage
- Chaque nœud envoie des mises à jour à (et reçoit des mises à jour de) ses voisins
  - périodiquement (de l'ordre de plusieurs secondes)
  - quand sa table change
- Chaque mise à jour est une liste de couples
  - <Destination, Coût> → vecteur de distance
- Mise à jour de la table si meilleure route
- Supprime les routes non confirmées pendant un certain temps

## Vecteur de Distance

- Soit  $D(i,j)$  la métrique de la meilleure route de  $i$  à  $j$
- Soit  $d(i,k)$  le coût pour aller directement de  $i$  à  $k$  si  $i$  et  $k$  sont voisins sinon  $d(i,k) = \infty$ 
  - Pour les nœuds non adjacents, la distance est supposée infinie
- Alors :
  - $D(i,i) = 0$  pour tout  $i$
  - $D(i,j) = \min_k [d(i,k) + D(k,j)]$

- Initialisation
  - Pour toutes les destinations  $y$ 

$$D(x,y) = d(x,y) \text{ /* Si } y \text{ n'est pas un voisin alors } \infty \text{ */}$$
  - Pour chaque voisin  $w$ 

$$D(w,y) = \infty \text{ pour toutes les destinations } y$$
  - Pour chaque voisin  $w$ 

$$\text{Envoyer le vecteur de distance de } x : [D(x,y)]$$
- Boucle
  - Attendre jusqu'à recevoir un vecteur de distance de  $w$  ou voir un changement sur une liaison vers un voisin  $w$
  - Pour chaque destination  $y$ 

$$D(x,y) = \min_w [d(x,w) + D(w,y)]$$
  - Si  $D(x,y)$  change pour une destination  $y$ 

$$\text{Envoyer vecteur de distance de } x \text{ à tous ses voisins}$$

Extrait Kurose Ross

## Exemple 1

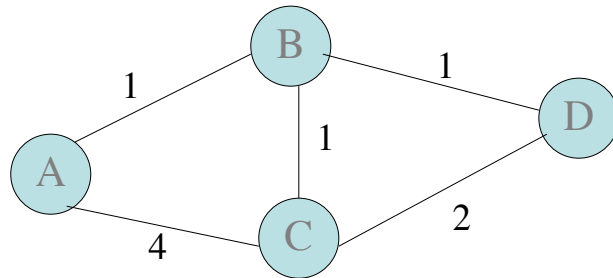


Table initiale pour chaque nœud ?

Table de routage au nœud A ?

17

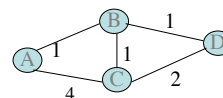
Table initiale pour A (démarrage à froid) :

| Destination | Coût | Prochain nœud |
|-------------|------|---------------|
| A           | 0    | -             |

A envoie le vecteur de distance  $\langle A, 0 \rangle$  à ses voisins B et C

A reçoit de ses voisins les vecteurs de distances  $\langle B, 0 \rangle$  et  $\langle C, 0 \rangle$

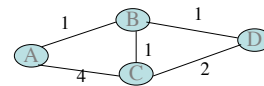
→ Mise à jour de la table



18

### Table initiale pour A (suite)

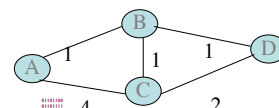
| Destination | Coût | Prochain nœud |
|-------------|------|---------------|
| A           | 0    | -             |
| B           | 1    | B             |
| C           | 4    | C             |



### Table initiale pour D

| Destination | Coût | Prochain nœud |
|-------------|------|---------------|
| B           | 1    | B             |
| C           | 2    | C             |
| D           | 0    | -             |

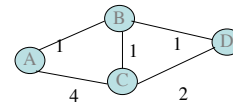
D envoie à C et B les vecteurs de distance  $\langle B, 1 \rangle$ ,  $\langle C, 2 \rangle$ ,  $\langle D, 0 \rangle$



## Table initiale pour B

| Destination | Coût | Prochain nœud |
|-------------|------|---------------|
| A           | 1    | A             |
| B           | 0    | -             |
| C           | 1    | C             |
| D           | 1    | D             |

B envoie à A,C et D les vecteurs de distance  
 $\langle A,1 \rangle$ ,  $\langle B,0 \rangle$ ,  $\langle C,1 \rangle$ ,  $\langle D,1 \rangle$



## Mise à jour de la table de A

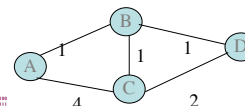
| Destination | Coût  | Prochain nœud |
|-------------|-------|---------------|
| A           | 0     | -             |
| B           | 1     | B             |
| C           | (4) 2 | (C) B         |
| D           | 2     | B             |

A reçoit de B les vecteurs de distance  $\langle A,1 \rangle$ ,  $\langle B,0 \rangle$ ,  $\langle C,1 \rangle$ ,  $\langle D,1 \rangle$

$$d(A,B) = 1$$

$$d(A,C) = 2$$

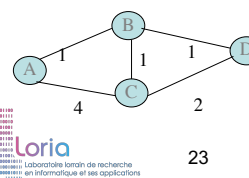
$$d(A,D) = 2$$



## Table initiale pour C

| Destination | Coût | Prochain nœud |
|-------------|------|---------------|
| A           | 4    | A             |
| B           | 1    | B             |
| C           | 0    | -             |
| D           | 2    | D             |

C envoie à A, B, et D les vecteurs de distance  
 $\langle A, 4 \rangle$ ,  $\langle B, 1 \rangle$ ,  $\langle C, 0 \rangle$ ,  $\langle D, 2 \rangle$

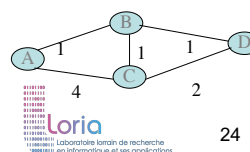


23

## Table de A

| Destination | Coût | Prochain nœud |
|-------------|------|---------------|
| A           | 0    | -             |
| B           | 1    | B             |
| C           | 2    | B             |
| D           | 2    | B             |

A reçoit de C les vecteurs de distance  $\langle A, 4 \rangle$ ,  $\langle B, 1 \rangle$ ,  $\langle C, 0 \rangle$ ,  $\langle D, 2 \rangle$



24

## Panne dans le réseau

- Que se passe-t-il quand un nœud tombe en panne ?
- Comment détecte-t-on une panne ?
- Le système reste-t-il dans un état cohérent ?

## Exemple 2

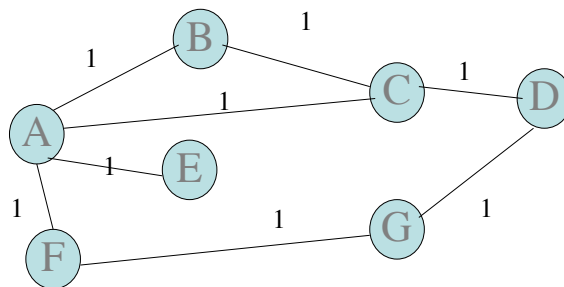
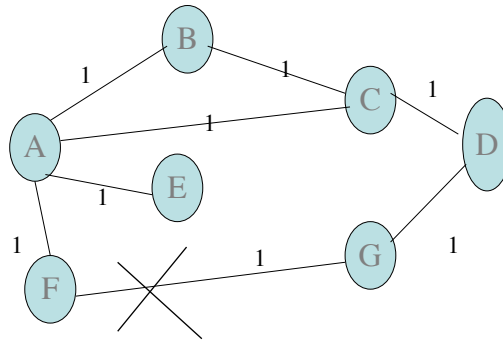
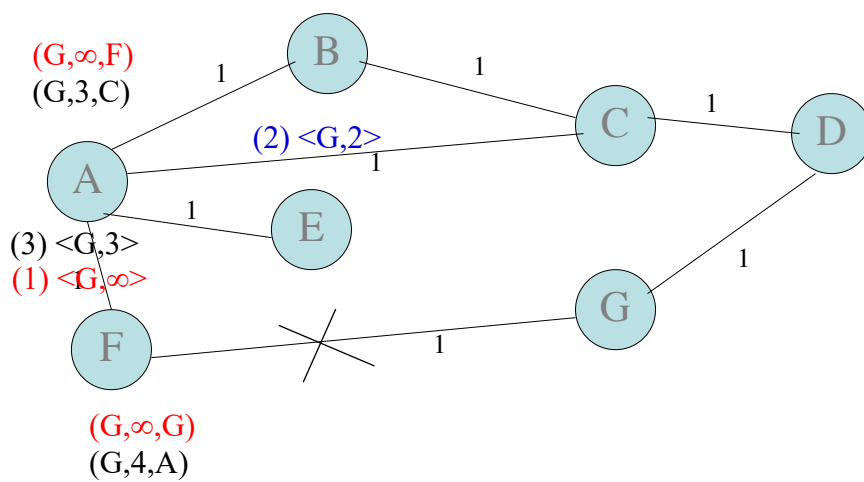


Table de routage au nœud A ?

- Que se passe-t-il si F détecte que la liaison vers G est en panne ?

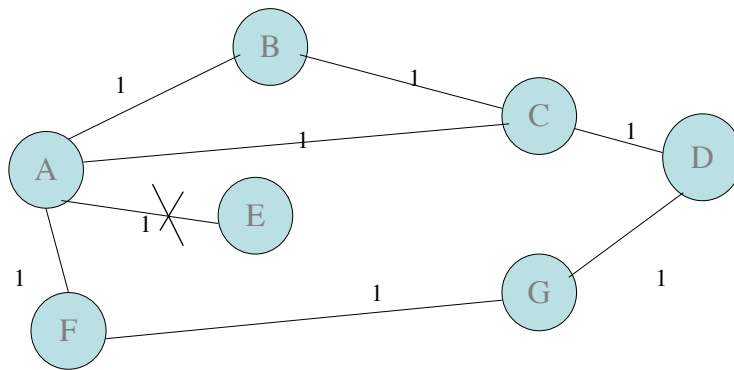


27

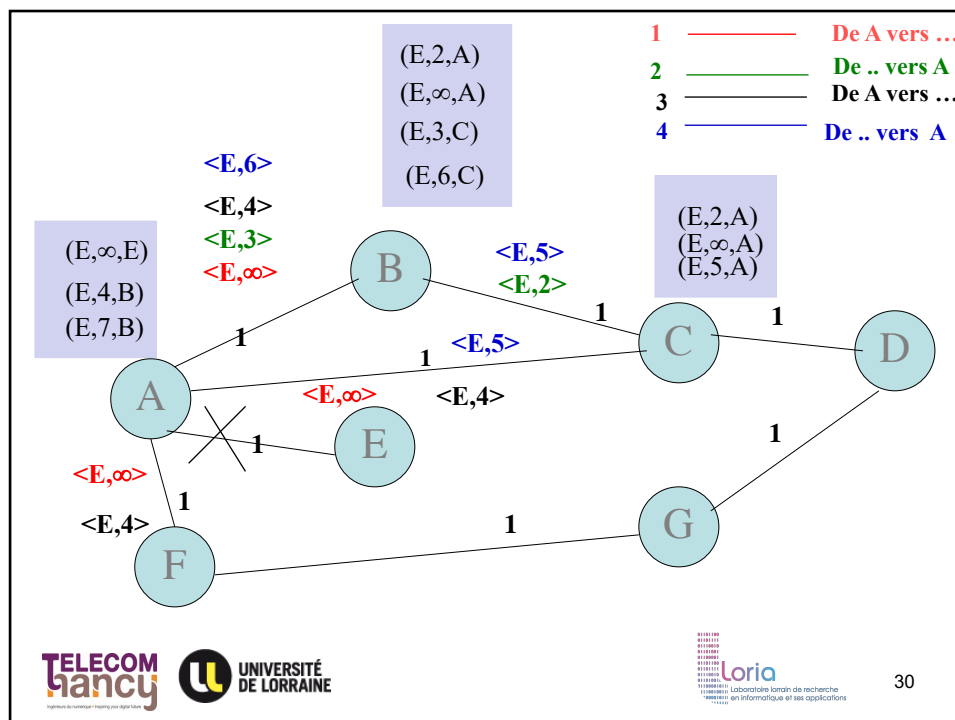


28

- Que se passe-t-il si la liaison de A vers E tombe en panne ?



29



30

## Heuristiques

- Représenter l'infini par une valeur finie : 16
- Vecteur de chemin
- L'horizon partagé
- Source tracing

## Vecteur de chemin

- Manque d'informations dans le vecteur
- Ex: pour aller de C à E , il faut passer par A
- Solution : Annoter chaque entrée avec le chemin pour obtenir le coût
  - $\langle E, 2, \text{« C-A-E »} \rangle$
- Les vecteurs nécessitent des tables assez grandes

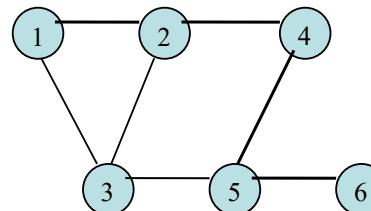
## Split Horizon Horizon partagé

- Un routeur n'annonce jamais la route à son voisin N, si N est le prochain nœud vers cette destination
  - C ne prévient pas A du coût vers E comme il utilise A comme prochain nœud
- L'horizon partagé omet les routes apprises d'un voisin dans les mises à jour envoyées à ce voisin

## Source tracing

- Le vecteur de distance contient le routeur précédent la destination

| Destination | Next | Last |
|-------------|------|------|
| 1           | -    | -    |
| 2           | 2    | 1    |
| 3           | 3    | 1    |
| 4           | 2    | 2    |
| 5           | 2    | 4    |
| 6           | 2    | 5    |



## Exemple de protocoles de routage à vecteur de distance

- RIP (Routing Information Protocol) est un protocole à vecteur de distance :
  - infini = 16, échange d'information toutes les 30s, routeur mort si silence pendant 180s, horizon partagé avec retour empoisonné
    - RIPv1 : statut historique (rfc1058)
    - RIPv2 : (rfc1723) inclut des mécanismes de sécurité

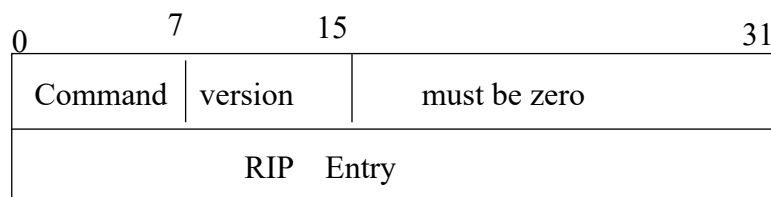
## Le protocole RIP

- Routing Information Protocol
  - Programme routé sous unix
- RIP version 2
  - RFC 2453 (1998)
- Implémentation du routage à vecteur de distance

## Le protocole RIP

- RIP est implémenté au dessus de UDP
  - Envoi et réception de paquets messages sur le port 520
- Deux types de messages
  - Messages d'information de routage
  - Messages pour demander l'information de routage
  - Même format général

## Message RIP



Commandes :

1: requête

2: réponse ou message non sollicité de mise à jour

Version :

1: RIP-1

2: RIP-2

RIP entry ; (1-25)

## Message RIP-1 entry

|                           |   |              |    |
|---------------------------|---|--------------|----|
| 0                         | 7 | 15           | 31 |
| Address family identifier |   | must be zero |    |
| IP address                |   |              |    |
| must be zero              |   |              |    |
| must be zero              |   |              |    |
| metric                    |   |              |    |

AFI : seul AF\_INET est supporté

Métrieque :

1-15 : métrieque courante pour la destination

16 ; infinie



39

39

## Message RIP-2 entry

|                           |   |           |    |
|---------------------------|---|-----------|----|
| 0                         | 7 | 15        | 31 |
| Address family identifier |   | Route Tag |    |
| IP address                |   |           |    |
| Subnet mask               |   |           |    |
| Next hop                  |   |           |    |
| metric                    |   |           |    |

Les messages RIP-2 sont diffusés sur une adresse IP multicast 224.0.0.9



40

40

## Message RIP-2 Authentication

- La partie authentification utilise une entrée entière de RIP
  - Si l'AFI de la première entrée (et seulement la première) est 0xFFFF alors le reste de l'entrée contient l'authentification
- Algorithmes d'authentification
  - KEYED-MD5, HMAC-SHA-1, HMAC-SHA-256, HMAC-SHA-384, and HMAC-SHA-512
- Authentication data
  - One-way function of a message and a secret RIP-2 Authentication Key

## Configuration RIPv1 Commandes CISCO

| Command                                          | Command's purpose                               |
|--------------------------------------------------|-------------------------------------------------|
| Rtr(config)#router rip                           | Enables RIP routing process                     |
| Rtr(config-router)#network                       | Associates a network with a RIP routing process |
| Rtr#debug ip rip                                 | used to view real time RIP routing updates      |
| Rtr(config-router)#passive-interface fa0/0       | Prevent RIP updates from going out an interface |
| Rtr(config-router)#default-information originate | Used by RIP to propagate default routes         |
| Rtr#show ip protocols                            | Used to display timers used by RIP              |

```

R1(config)#router rip
R1(config-router)#network 192.168.1.0
R1(config-router)#network 192.168.2.0

R2(config)#router rip
R2(config-router)#network 192.168.2.0
R2(config-router)#network 192.168.3.0
R2(config-router)#network 192.168.4.0

R3(config)#router rip
R3(config-router)#network 192.168.4.0
R3(config-router)#network 192.168.5.0

```

TELECOM nancy UNIVI DE LORRAINE Loria

## Configuration RIPv2

### Commandes CISCO

| Command                                          | Command's purpose                               |
|--------------------------------------------------|-------------------------------------------------|
| Rtr(config)#router rip                           | Enables RIP routing process                     |
| Rtr(config-router)#version 2                     | Enables Version 2                               |
| Rtr(config-router)#no auto-summary               | To disable automatic summarization              |
| Rtr(config-router)#network                       | Associates a network with a RIP routing process |
| Rtr#debug ip rip                                 | used to view real time RIP routing updates      |
| Rtr(config-router)#passive-interface fa0/0       | Prevent RIP updates from going out an interface |
| Rtr(config-router)#default-information originate | Used by RIP to propagate default routes         |
| Rtr#show ip protocols                            | Used to display timers used by RIP              |

TELECOM nancy UNIVERSITÉ DE LORRAINE Loria

# Protocoles de Routage : *Open Shortest Path First (OSPF)*

Rémi Badonnell  
Slides d'Isabelle Chrisment

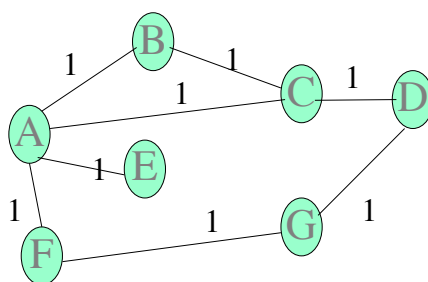
## Algorithme de Routage État de liaisons – Link State Routing

- Stratégie: envoyer à tous les nœuds (pas seulement les voisins) les informations au sujet de ses voisins
- Distribution de la topologie du réseau et du coût de chaque liaison à tous les routeurs

## État de liaisons

- Les nœuds ont une copie complète de la carte du réseau
- Les nœuds exécutent le calcul des meilleures routes localement en utilisant cette carte
  - Plus de boucles

## Carte de réseau



|       |   |   |
|-------|---|---|
| A     | B | 1 |
| A     | C | 1 |
| A     | E | 1 |
| A     | F | 1 |
| B     | A | 1 |
| B     | C | 1 |
| C     | A | 1 |
| C     | B | 1 |
| C     | D | 1 |
| ..... |   |   |

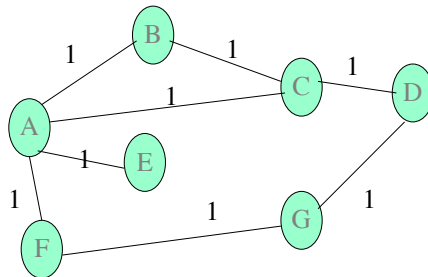
## État de liaison

- Basé sur 2 mécanismes
  - Dissémination fiable de l'information (reliable flooding)
    - Pour permettre une mise à jour de la base de données
  - Calcul des routes à partir des informations locales

## Dissémination de la topologie

- Chaque routeur met les informations décrivant les liaisons dans des paquets appelés LSP (Link State Packet) ou LSA (Link State Advertisement)
- LSA :
  - L'ID ou identificateur du nœud qui a créé le LSA
  - Une liste des voisins directement connectés avec le coût de la liaison vers chaque voisin
  - numéro de séquence (SEQNO)
  - time-to-live (Age) pour ce paquet

## LSA



|   |   |   |
|---|---|---|
| A | B | 1 |
| A | C | 1 |
| A | E | 1 |
| A | F | 1 |

## Inondation ( Flooding) Algorithme

- Recevoir un LSA et chercher l'enregistrement correspondant dans la base
- Si enregistrement pas présent
  - Ajouter et diffuser le LSA à tous les nœuds excepté celui de la réception
- Si enregistrement présent
  - Comparer le numéro de séquence du paquet reçu avec celui stocké
  - Si le numéro de séquence dans la base est plus **ancien** que celui du LSA reçu, diffuser le LSA à tous les nœuds excepté celui de la réception

## Calcul de la route en théorie

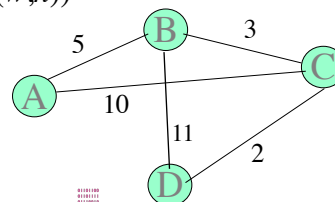
- Algorithme utilisé :
  - Algorithme du plus court chemin d'abord de Dijkstra (Short Path First)
  - Théorie des graphes
    - Soit  $N$  l'ensemble des nœuds dans le graphe
    - $l(i,j)$  poids sur l'arc entre le nœud  $i$  et  $j$ . Si aucun arc entre  $i$  et  $j$  alors  $l(i,j) = \infty$
    - $C(n)$  détermine le coût du chemin de  $s$  vers le nœud  $n$

$M = \{s\}$   
 Pour tout  $n$  dans  $N - \{s\}$   
 $C(n) = l(s,n)$

Tant que  $(N \neq M)$

$M = M \cup \{w\}$  tel que  $C(w)$  est le minimum pour tout  $w$  dans  $(N-M)$

Pour tout  $n$  dans  $(N-M)$   
 $C(n) = \text{MIN} (C(n), C(w) + l(w,n))$



## Calcul de la table de routage en pratique

- Mise en œuvre de l'algorithme de Dijkstra
- Chaque routeur maintient deux listes
  - Temporaire (T) et Permanent (P)
- Chaque liste contient 3 éléments
  - Destination, Coût, Prochain nœud

1. Initialiser la liste P avec une entrée pour le nœud lui-même ( $s$ ). Cette entrée a un coût de 0
2. Pour le nœud ajouté dans la liste P lors de la précédente étape et appelé *Next*, sélectionner son LSP (si 1<sup>ère</sup> fois  $Next < s$ )
3. Pour chaque *Voisin* de *Next*, calculer le coût pour atteindre son *Voisin* comme la somme du coût de  $s$  à *Next* avec celui de *Next* au *Voisin*.

Deux cas peuvent se produire :

a) Si *Voisin* n'est présent ni dans la liste P ni dans la liste T **alors**

- ajouter <*Voisin*, Coût, Nexthop> dans la liste T, avec *Nexthop* étant la direction que *s* doit prendre pour atteindre *Next* (ou *Voisin* si 1<sup>ère</sup> fois)

b) Si *Voisin* est présent dans la liste T et le coût inférieur à celui actuellement listé pour *Voisin*, **alors**

- remplacer l'entrée courante par <*Voisin*, Coût, Nexthop> où Nexthop est la direction que *s* doit prendre pour atteindre Next

4. Si la liste T est vide **alors**

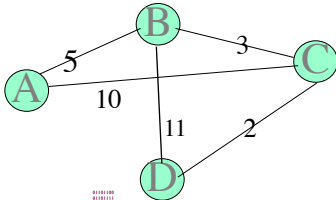
- arrêter

**sinon**

- prendre une entrée dans la liste T avec le coût le plus faible, la déplacer dans la liste P et retourner en 2




57

| Etape | Permanent                                                                                  | Temporaire                                                                           |
|-------|--------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| 1.    | <p>(D, 0, -)</p> <ul style="list-style-type: none"> <li>•</li> <li>•</li> <li>•</li> </ul> |  |




58

## Exemples de protocoles de routage à état de liaison

- OSPF (Open Shortest Path First Protocol) : protocole à état de liaison recommandé pour remplacer RIP
  - Load-balancing
  - Authentification
  - Hiérarchie : domaines sont partitionnés en area

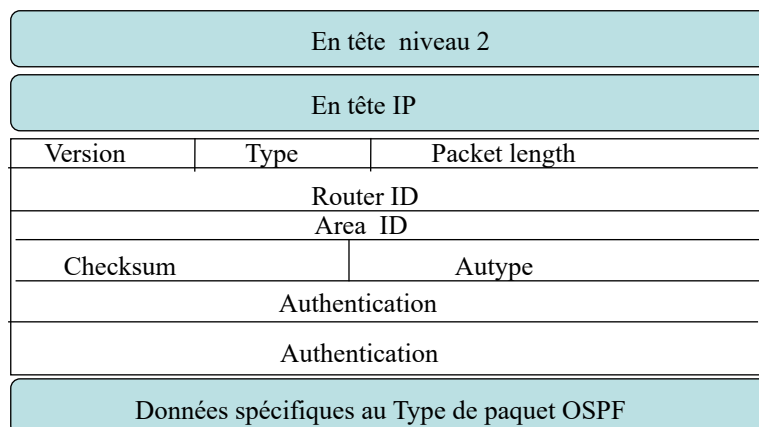
## OSPF Routage hiérarchique

- Le routage est hiérarchisé pour simplifier le calcul des routes
  - Découpage en AREA
- Une area est un ensemble de réseaux contigus
- Chaque AREA se comporte comme un réseau indépendant

## OSPF

- RFC 2328 (Avril 1998)
- Fonctionne directement au dessus de IP (protocole type 89)
- Est composé de 3 sous-protocoles
  - Hello
    - Hello Packet
  - Exchange
    - Database Description (DD)
  - Flooding/Inondation
    - Link State Request
    - Link State Acknowledgment
    - Link State Update
- Tous les paquets OSPF commencent avec un en-tête commun

## En-tête OSPF



## OSPF

### Le protocole d'inondation

- Mise à jour de la base de données
  - Emission d'un LSA pour demander des informations spécifiques
    - Link State Request (Type OSPF 3)
  - Transmission des informations
    - Link State Update (Type OSPF 4)
- Quand une liaison change d'état
  - Émission d'un LSA pour informer du changement d'état
    - Link State Update (Type OSPF 4)
  - Acquiescement pour fiabiliser le protocole d'inondation
    - Link State Acknowledgment (Type OSPF 5)

## Configuration OSPF - Commandes CISCO

| Command                                                                                       | Command's purpose                                                                   |
|-----------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Rtr(config)#router ospf <i>process-id</i>                                                     | Enables OSPF routing process                                                        |
| Rtr(config-router)#network <i>network-address</i><br><i>wildcard-mask</i> area <i>area-id</i> | Associates a network with an OSPF routing process                                   |
| Rtr(config-router)#router-id <i>ip-address</i>                                                | To force Router ID                                                                  |
| Rtr(config-router)#passive-interface <i>fa0/0</i>                                             | Suppress the sending of hello packets through the specified interface.              |
| Rtr(config-router)#default-information originate                                              | Used by OSPF to propagate default routes                                            |
| Rtr#show ip protocols                                                                         | To display the parameters and current state of the active routing protocol process, |
| Rtr#show ip ospf                                                                              | Used to display general information about OSPF                                      |
| Rtr#show ip ospf database                                                                     | To display lists of information related to the OSPF database                        |
| Rtr#show ip ospf neighbor                                                                     | To check neighbor relationships                                                     |
| Rtr#debug ip ospf event<br>Rtr#debug ip ospf packets                                          | used to view real time OSPF events/packets                                          |

## Choisir le coût des liaisons

- Déterminer la façon dont la charge de trafic est distribuée dans le réseau
  - Plus le coût est faible pour une liaison, plus la probabilité de choisir cette liaison dans un plus court chemin est grande
- Différentes métriques possibles
  - Métrique statique
  - Métrique dynamique de l'ARPAnet
  - Métrique dynamique de l'ARPAnet modifiée

## Métrique statique

- Le plus simple donne un poids de 1 à chaque liaison
  - Le chemin le plus court est le chemin avec le moins de routeurs intermédiaires
  - Problème ?
- Assigner un poids différent en fonction du type de liaison
  - Problème ?

# Module Réseaux et Systèmes Avancés (RSA) – Partie Réseau

Rémi Badonnel  
Slides d'Isabelle Chrisment



67